Lab Manual 08 Threads

61 WHAT IS A THREAD ?

"A thread is a sequence of control within a process"

To the software developer, the concept of a "procedure" that runs independently from its main program may best describe a thread.

62 PROPERTIES OF THREADS

Threads use and exist within process resources, yet are able to be scheduled by the operating system and run as independent entities largely because they duplicate only the bare essential resources that enable them to exist as executable code.
This independent flow of control is accomplished because a thread maintains its own:

1. Stack pointer
2. Registers
3. Scheduling properties (such as policy or priority)
4. Set of pending and blocked signals
5. Thread specific data

63 SUMMARIZE THREADS

1. Exists within a process and uses the process resources.
2. Has its own independent flow of control as long as its parent process exists and the OS supports it.
3. Duplicates only the essential resources it needs to be independently schedulable.
4. May share the process resources with other threads that act equally independently (and dependently).
5. Dies if the parent process dies.

6. Is "lightweight" because most of the overhead has already been accomplished through the creation of its process.

7. Threads do Not Share:

(a) Thread ID

(b) Set of registers, including program counter and stack pointer

(c) Stack (for local variables and return addresses)

(d) Signal mask

(e) Priority

64 THREAD IMPLEMENTATION

In order to take full advantage of the capabilities provided by threads, a standardized programming interface was required. For UNIX systems, this interface has been specified by the IEEE POSIX 1003.1c standard (1995).

Implementations adhering to this standard are referred to as POSIX threads, or **Pthreads**. Pthreads are defined as a set of C language programming types and procedure calls, implemented with a **pthread.h** header

To link this library to your program use
gcc −pthread −o a . out myprogram . c

65 THREAD CREATION

Initially, your main() program comprises a single, default thread. All other threads must be explicitly created by the programmer.

pthread_create creates a new thread and makes it executable. This routine can be called any number of times from anywhere within your code.

```
int pthread_create ( pthread_t *thread , const void *(* pthread_attr_t * attr ,
start_routine ) ( void *) , void *arg ) ; PTHREAD_CREATE
```

ARGUMENTS:

1. **thread:** unique identifier for the new thread returned by the subroutine.

2. **attr:** An attribute object that may be used to set thread attributes. You can specify a thread attributes object, or NULL for the default values.

3. **start_routine:** The C routine that the thread will execute once it is created.

4. **arg:** A single argument that may be passed to start_routine. It must be passed by reference as a pointer cast of type void. NULL may be used if no argument is to be passed.

66 PTHREAD FUNCTIONS

PTHREAD_SELF()

Return the thread ID of currently executing thread. Returns

```
pthread_t pthread_self ( void ) ;
```

the unique thread ID of the calling thread.

PTHREAD_EXIT()

Used to terminate threads

```
pthread_exit (* status )
```

This is also the mechanism used to get a return value from a thread. Several ways of termination:

1. The thread returns from its starting routine (the main routine for the initial thread).

2. The thread makes a call to the pthread_exit subroutine.

3. The thread is canceled by another thread via the pthread_cancel routine.

4. The entire process (main) is terminated due to a call to the exit subroutines.

EXAMPLE 01

```
# include <s t d i o . h>                    object of type " pthread_t " //for storing
# include <s t d l i b . h>                      thread id
#  include <pthread . h> pthread_t thread_id ;
//creating

void printids ( const char *s ) {
  pid_t pid ; pthread_t
  tid ; pid = getpid ( ) ;
   tid =  pthread_self ( ) ;
```

```
        printf ( "%s pid %lu tid %lu\n" , s , ( unsigned long ) pid , ( unsigned long ) tid ) ;
    }

void * thr_fn ( void *arg ) {                    // New thread    will    c a l l    this function/routine .
            printids ( "New thread :        " ) ;
          return (NULL) ;
    }

 int        main( void ) { int
                ret ;
            /* create                                          a          thread , store      i t s        id in
                                                               thread_id with default      a t t r i b  by
                                                               passing NULL*/
            /* Thread will immediately execute " thr_fn " function with NULL parameters */
            ret = pthread_create(&thread_id , NULL, thr_fn , NULL) ; if ( ret != 0) printf ( " can '
            t create thread\n" ) ;
              printids ( "Main        thread : " ) ;
           /* Exit from main ( ) will cause new thread to destroy , l e t s give i t some time
            */ sleep ( 1 ) ;
            exit ( 0 ) ;
}
```

71 JOINING THREADS

```
int      pthread_join        ( pthread_t tid ,      void ** status ) ;
```

1. "Joining" is one way to accomplish synchronization between threads.

2. The pthread_join() subroutine blocks the calling thread until the specified threadid thread terminates.

3. The programmer is able to obtain the target thread's termination return status if it was specified in the target thread's call to pthread_exit().

4. A joining thread can match one pthread_join() call. It is a logical error to attempt multiple joins on the same thread.

```
                <s t d i o . h>
# include < s t d l i b . h> # include <pthread . h>

void * thr_fn1 ( void *arg ) { printf ( " Thread 1 exiting\n" ) ;
            void * status=" Success " ;
            pthread_exit ( ( void *) status ) ;
}
```

EXAMPLE 02

```
# include

void * thr_fn2 ( void *arg ) { printf ( " Thread 2
        exiting\n" ) ; pthread_exit ( ( void * ) 2 ) ;
}

int

    main( void ) { pthread_t tid1 , tid2 ; pthread_create(&tid1 , NULL,
        thr_fn1 , NULL) ; pthread_create(&tid2 , NULL, thr_fn2 , NULL)
        ; void * tret1 ; void * tret2 ;
/*          why passing    adress   of  pointer     tret1  ?  see          Pthread_join  prototype ! !    */
        pthread_join ( tid1 , &tret1 ) ; pthread_join ( tid2 ,
        &tret2 ) ;
/*          casting    tret1   to   char*                                                              */

        printf ( " Thread 1 exit code is : %s\n" , ( char *) tret1 ) ; printf ( " Thread 2
        exit code is : %ld\n" , ( long ) tret2 ) ; exit ( 0 ) ;

}
```

72 **PASSING ARGUMENTS TO THREADS**

The pthread_create() routine permits the programmer to pass one argument to the thread start routine.

For cases where multiple arguments must be passed, this limitation is easily overcome by creating a structure which contains all of the arguments, and then passing a pointer to that structure in the pthread_create() routine.

All arguments must be passed by reference and cast to (void *).

EXAMPLE 03

```
# include     <s t d i o . h>
# include     < s t d l i b . h>
# include      <pthread . h>
/* this function is run by the second thread */ void *inc_x ( void *arg ) { /* increment x to 100 */ printf ( "
Thread : Process Id is %d and Thread Id is %ld\n" , getpid ( )
                                                                                , pthread_self ( ) ) ;
            int *input ;
            input =( int *) arg ; while (++(* input
             )          <          100);
                printf ( "x      increment      finished\n" ) ;
            /* the function must return something – NULL will do */ pthread_exit (NULL) ;
            }
void main ( ) { // main Thread ; int x = 0 , y = 0;
```

```
        /* show the i n i t i a l values of x and y */ printf ( "x : %d , y : %d\n"
        , x , y ) ;
          printf ( "main :    Process  Id        is %d and Thread        Id      is %ld\n" , getpid ( )
                                                                                , pthread_self ( ) ) ; /*
        this variable is our reference to the second thread */ pthread_t thread_2 ;

          /* create    a  second    thread    which    executes    inc_x(&x ) */
        ( pthread_create(&thread_2 , NULL, inc_x , &x ) ) ; /* increment y
        to 100 in the f i r s t thread */ while(++y < 100);
          printf ( "y    increment    finished\n" ) ;
            /* wait for the    second    thread  to    finish */
          ( pthread_join ( thread_2 , NULL) ) ;

          /* show the results - x is now 100 thanks to the second thread */ printf ( "x : %d , y : %d\n" , x ,
          y ) ; pthread_exit (NULL) ;

}
```

EXAMPLE 04

```
# include    <pthread . h>
# include <s t d i o . h> # include
        < s t d l i b . h> struct
        student_database {
          int    rollno ; f
          l o a t   cgpa ;
} ;

struct      student_database    obj ;

void * print_fn ( void *threadarg ) { struct student_database
          *temp_data ;
            temp_data =    ( struct      student_database *)      threadarg ;
          int rollno= temp_data->rollno ; f l o a t cgpa = temp_data->cgpa ;
          printf ( " Inside Thread\n" ) ; printf ( " Roll no %d has cgpa :% f\n" ,
          rollno , cgpa ) ; pthread_exit (NULL) ;
}

int    main( int    argc ,    char *argv [ ] ) {
          pthread_t thread_id ; obj . rollno =90010; obj . cgpa =3.88; printf ( "
          Creating thread\n" ) ; pthread_create(&thread_id , NULL, print_fn ,
          &obj ) ; pthread_exit (NULL) ;
}
```

EXAMPLE 05

```
# include      <pthread . h>
# include <s t d i o . h> # include < s t d l i b . h>

pthread_t thread_id1 , thread_id2 ;

void * thr_fn1 ( void *threadid ) { sleep ( 1 ) ; printf ( " thread          1:        #%ld !  and Iam  not
                    waiting\n" , pthread_self ( ) ) ; int       i =0;
            for ( i ; i <5; i ++){ printf ( " value of i is %d\n" , i ) ; } pthread_exit (NULL) ;
}

void * thr_fn2 ( void *threadid ) { printf ( " thread 2: #%ld ! waiting for thread 1\n" , pthread_self ( ) ) ;
            pthread_join ( thread_id1 ,NULL) ; printf ( " thread 2: thread 1 exited and now me exiting \n"
            ) ; pthread_exit (NULL) ;
}

int  main(  int  argc  ,  char  *argv  [  ]  )  {  printf  (  "main  :  creating  thread\n"  )  ;
            pthread_create(&thread_id1    ,    NULL,    thr_fn1    ,NULL)    ;
            pthread_create(&thread_id2 , NULL, thr_fn2 ,NULL) ; pthread_join (
            thread_id2  ,NULL)  ;  printf  (  "main  :  Both  threads  exited\n"  )  ;
            pthread_exit (NULL) ;
}
```

EXAMPLE 06

```
# includ        <pthread . h>
# includ          <s t d i o . h>
        ne NUM_THREADS 5

void     * PrintHello ( void *threadid ) { long * tid ;
              tid = ( long *) threadid ; printf ( " It ' s me,
              thread #%ld\n" , pthread_exit (NULL) ;
                                                          * tid ) ;
}

int
        main ( int argc , char *argv [ ] ) { pthread_t threads
              [NUM_THREADS] ; long t ;
              for ( t =0;    t <NUM_THREADS;    t ++){ printf ( "
                          In          main :     creating          thread %ld\n" ,        t ) ;
                      pthread_create(&threads [ t ] , NULL, PrintHello , sleep ( 1 ) ;          ( void    *)& t ) ;
}

/* Last thing that main ( ) pthread_exit should do */ (NULL) ;


}
```

EXAMPLE 07

```
# include        <pthread . h>
# include <s t d i o . h> #
include     < s t d l i b . h>
char *message ;
struct          thread_data
{
       pthread_t *thread_id ; char
      *message ;
} ;
struct          thread_data         thread_data ;

void * thr_fn ( void *threadarg ) { struct thread_data
                *my_data ;
                  my_data =       ( struct            thread_data *) threadarg ;
                pthread_t * taskid              = my_data->thread_id ; char
                *hello_msg= my_data->message ;
                printf ( " Thread id is : %ld \nMessage : %s " , *taskid , hello_msg ) ; pthread_exit ( ( void *) "
                Successfull " ) ;
}

int      main( int       argc ,        char *argv [ ] ) {
                  pthread_t       thread_id1 ;
                  message =       "Have you      learn      something   today   ?\n" ;
                thread_data . thread_id =&thread_id1 ; thread_data . message = message ; printf ( " Creating
                thread\n" ) ; pthread_create(&thread_id1 , NULL, thr_fn , ( void *) &thread_data ) ;
                void * status ;
                pthread_join ( thread_id1 ,& status ) ; printf ( " Thread Exited with status : %s\n"
                ,( char *) status ) ; exit ( 0 ) ;
}
```